Replication Report: Machine Learning and the Implementable Efficient Frontier (Jensen, Kelly, Malamud & Pedersen, 2026)

2026-10-08

QF600 Asset Pricing · Group project report · Singapore Management University

Abstract

This report replicates Jensen, Kelly, Malamud and Pedersen (2026), "Machine Learning and the Implementable Efficient Frontier" (*Review of Financial Studies* 39: 3035–3078). The paper proposes Portfolio-ML, which learns portfolio weights directly from stock characteristics by maximising mean–variance utility net of trading costs, and evaluates portfolio methods by their out-of-sample, net-of-cost risk and return. We port the authors' R code to Python and rerun the method on the 80 largest US stocks with 18 characteristics, testing on January 2020 – December 2025, a period after the original sample.

The replication reproduces the paper's qualitative results. Cost-blind ML portfolios (Markowitz-ML, Factor-ML) are unimplementable after trading costs; cost-aware methods cut costs by two or more orders of magnitude; Portfolio-ML ranks first among cost-aware methods in utility; and return correlations between methods are close to the paper's. The paper's performance levels are not reproduced: Portfolio-ML earns a net Sharpe ratio of 0.03 against 1.33, and its edge over Static-ML* is not statistically significant (65% against 99.9%). The evidence points to the near absence of return predictability within a mega-cap universe (forecast rank correlation 0.002, against 0.07 across all US stocks) rather than to implementation error.

1. Introduction

1.1 Objective

The objective of this project is to replicate the core empirical results of Jensen et al. (2026) and to assess how closely they hold in a different setting. The original study tests its methods on about 650 NYSE stocks with 115 characteristics over 1981–2020. We rebuild the full pipeline, from raw data to the paper's tables and figures, at a scale that runs on a laptop, and test it on data the authors did not use.

1.2 Scope

We replicate eight of the eleven methods in the paper's main performance table and six of its principal outputs: Table 2 (out-of-sample performance), Figure 2 (performance over time), Table 3 (statistical significance), Table 4 (return correlations), Figures 1A and 1B (the implementable efficient frontier) and Figure 5 (economic feature importance). Multiperiod-ML, Rank-ML, short-selling fees, size-group analysis and the simulation study are outside our scope. Table 1.1 summarises the outcome.

Table 1.1 – Summary of replicated outputs and their agreement with the original study.

Paper output	What it shows	Replicated?	How closely it matches
Table 2	Out-of-sample performance by method, net of costs	Yes, 8 of 11 rows	**Structure matches, levels do not.** Cost-blind methods collapse after costs, as in the paper. Portfolio-ML > Static-ML* > Static-ML in utility, as in the paper. But Portfolio-ML's net Sharpe ratio is 0.03 vs 1.33, and every ML method has negative utility
Figure 2	Cumulative gross, net and utility returns over time	Yes	Partly. Gross and net lines coincide for cost-aware methods, as in the paper, but our curves are flat rather than rising
Table 3	Probability that one method beats another	Yes	Partly. 100% against Factor-ML and Markowitz-ML, as in the paper; 65% against Static-ML* vs 99.9%
Table 4	Correlation of gross returns between methods	Yes	**Close** for Portfolio-ML/Static-ML* (0.54 vs 0.52) and Static-ML*/Markowitz-ML (0.61 vs 0.58); weaker for Factor-ML pairs
Figure 1A	Implementable efficient frontier by method	Yes	**Shape matches** for cost-blind methods (frontiers dive below zero). Cost-aware frontiers are flatter and lower
Figure 1B	Portfolio-ML frontier by investor size	Yes	**Opposite direction.** The paper's frontier shrinks with wealth; ours does not
Figure 5	Economic feature importance by theme	Yes	Different themes matter (low risk, reversal, value vs quality, momentum, investment); results sensitive to the random shuffle
Multiperiod-ML(*), Rank-ML, Figures 3–4, size groups, shorting fees, simulations	—	No	Out of scope

In short, the mechanism the paper describes is reproduced, while its profitability is not.

1.3 Structure of this report

Section 2 summarises the original study and the finance and machine-learning concepts it relies on. Section 3 describes the data and asset universe, and Sections 4 and 5 the methodology: the model inputs and the portfolio methods. Section 6 describes how the replication is implemented and organised. Section 7 defines the evaluation metrics, Section 8 compares our results with the paper's, and Section 9 discusses why they differ. Section 10 concludes. The appendices list every deviation from the paper and the authors' code.

2. The original study

The paper argues that a portfolio method should be judged by the risk and return it delivers after trading costs, out of sample, and proposes a method, Portfolio-ML, that learns weights with those costs built in.

2.1 Key finance concepts

Mean–variance utility. The investor scores a portfolio by its expected excess return minus a penalty for variance, scaled by relative risk aversion γ. Utility is a certainty equivalent: the risk-free return the investor would accept instead. With $\gamma = 10$, a portfolio with 5% expected return and 10% volatility has utility $0.05 - \frac{1}{2} \cdot 10 \cdot 0.01 = 0$.

Price impact and quadratic trading costs. Buying a stock pushes its price up in proportion to the size of the trade relative to the stock's trading volume. The cost per dollar traded therefore rises with the trade size, so total

cost is quadratic in the trade. Two consequences follow: costs grow with fund size (for the same percentage trade, a $10B fund pays 100 times the dollar cost of a $1B fund, or 10 times as a share of its assets), and it pays to split a large trade over several months.

Signal persistence (alpha decay). A signal's value to a large investor depends on how long it keeps predicting returns. Book-to-market changes slowly (monthly autocorrelation 0.94, per the paper), so a value position can be built gradually and held. One-month reversal flips sign every month (autocorrelation −0.04), so profiting from it requires constant trading.

The efficient frontier, textbook versus implementable. The textbook frontier is the set of best risk–return combinations without costs: a straight line from the risk-free rate, along which the Sharpe ratio is constant. The *implementable* frontier is drawn from out-of-sample results net of costs. It bends down, because more risk means larger positions and larger trades, which cost disproportionately more.

Aim portfolio and partial adjustment (Gârleanu and Pedersen, 2013). With quadratic costs and persistent signals, the optimal strategy is to trade each month part of the way from the current portfolio towards an "aim" portfolio. The aim is a weighted average of the best portfolios for current and expected future signals. Higher costs mean slower trading towards it.

2.2 Key machine-learning concepts

Random Fourier features. A way to fit nonlinear functions with linear regression. Each stock's characteristic vector s is projected onto random directions w_j, and $\sin(s^T w_j)$ and $\cos(s^T w_j)$ become new regressors. A ridge regression on enough of them approximates any smooth function of s; it is equivalent to a two-layer neural network whose first layer is random and fixed.

Ridge regression. Least squares with a penalty λ on the squared size of the coefficients. The penalty shrinks coefficients towards zero, trading a little bias for much less variance, which matters when predictors are many and noisy.

One-stage versus two-stage learning. The standard approach first learns expected returns (minimising forecast error), then optimises a portfolio. Portfolio-ML learns the portfolio weights in one stage by maximising realised utility net of costs, so slow, cheap-to-trade signals are favoured automatically.

2.3 The claims we test

1. Cost-blind ML portfolios (Markowitz-ML, Factor-ML) have strong gross performance but negative net performance for a $10B investor.
2. Portfolio-ML delivers the highest net Sharpe ratio (1.33) and utility (0.086) of all methods.
3. Portfolio-ML beats Static-ML*, a sophisticated two-stage method with an extra tuning layer, with 99.9% probability.
4. The implementable frontier bends down and contracts as investor wealth grows.
5. After costs, persistent themes (quality, momentum, investment) matter most and short-term reversal matters little.

3. Asset universe and data

Our universe is the 80 largest US common stocks, against the paper's ~650 NYSE stocks above median size. That is the single biggest design difference: we test the method where trading is cheapest and anomalies are weakest.

3.1 Universe: paper versus replication

Table 3.1 – Investment universe: original study versus replication.

	Paper	**Replication**
Exchanges	NYSE only	NYSE + Nasdaq
Size filter	Above NYSE median market cap	Top 80 by market cap
Stocks per month	min 184, median 646, max 805	75–86 (80–86 at each reset)
Share codes	Common stocks (10, 11, 12)	Same; plus JKP flags `common = primary_sec = obs_main = 1` (one row per firm)
Data screens	Non-missing market cap, return, dollar volume, SIC; ≥ 57 of 115 features	Same, plus return at t+1; ≥ 9 of 18 features
History requirement	Signals for the current and past 11 months	Same (13 consecutive screened months)
Entry/exit rule	Enter after 12 months passing screens, leave after 12 months failing	Rank buffer, reset each December: members stay while ranked ≤ 100; non-members enter if ranked ≤ 70; fill to 80 by rank
Name turnover	Not reported	6% a year on average (1.2–15%)

Why NYSE + Nasdaq. The paper drops Nasdaq because Nasdaq stocks entered CRSP in 1972, creating artificial jumps in the universe during its sample. Our data starts in 1993, after that jump, and a mega-cap universe without Apple, Microsoft, Amazon, Nvidia or Alphabet would not be representative.

Why a rank buffer. Both rules exist to stop stocks flickering in and out, which would force trades and costs that no real manager would make. The paper's 12-month rule does this through time; our 70/100 buffer does it through rank. Selling a stock that leaves the universe is charged at that month's trading cost (the R code lets it leave free).

What this universe means economically. These 80 stocks are the most liquid in the world: Apple traded about $12 billion a day in December 2025. Trading costs per dollar are therefore far lower than in the paper's universe, but so is the return predictability that ML signals exploit, which is concentrated in smaller, less-followed stocks.

3.2 Sample periods

Table 3.2 – Sample periods.

	Paper	**Replication**
Data start	December 1952	January 1993
Return-model predictions	1971–2020	2015–2025
Hyperparameter validation starts	1971	2015
Out-of-sample test	1981–2020 (480 months)	2020–2025 (72 months)
Overlap with paper's test	—	12 months (2020)

Our config labels 2005–2014 "train" and 2015–2019 "validation", but as in the paper, every model and hyperparameter is re-estimated each year on an expanding window, so 2015–2019 is simply the first stretch of that walk-forward process.

3.3 Data sources

Table 3.3 – Data sources.

Data	Paper	Replication
Characteristics, monthly returns, market cap, dollar volume	JKP Global Factor Data (Jensen, Kelly and Pedersen, 2023)	Same, WRDS `contrib.global_factor`, US, 1993–2025
Daily returns for the risk model	`usa_dsf.csv`, built by JKP's SAS code (not public)	CRSP v2 daily file `crsp.dsf_v2`, total returns minus French daily risk-free rate
Market return	JKP `market_returns.csv`	Ken French Mkt-RF + RF (same concept: value-weighted CRSP market)
Risk-free rate	Ken French	Same
Feature themes and signs	JKP `Cluster Labels.csv`, `Factor Details.xlsx`	Same files

After screens the panel holds 1.88 million stock-months; the screens drop 4.6% of rows. We verified the units that drive trading costs before using them: dollar volume is in dollars (Apple, December 2025: $\$1.23 \times 10^{10}$ a day) and market cap in $ millions. A factor-of-10^6 error here would make costs zero or absurd.

3.4 Characteristics

The paper uses 115 characteristics grouped into 13 themes. We use 18 in 6 themes, three per theme, all among the paper's 115 and assigned to themes using JKP's own cluster labels:

Table 3.4 – Characteristics used in the replication, by theme.

Theme	Characteristics	Persistence
Value	book-to-market, sales-to-market, EBITDA-to-EV	Slow
Momentum	12-1 month return, 6-1 month return, price to 52-week high	Medium
Quality	QMJ profitability, QMJ safety, gross profits to assets	Slow
Investment	asset growth, PP&E + inventory growth, net operating asset growth	Slow
Low risk	idiosyncratic volatility, 60-month beta, 21-day volatility	Medium
Short-term reversal	1-month return, 21-day return skewness, 21-day idiosyncratic skewness	Fast

The selection keeps both slow and fast signals, so the paper's central contrast (persistent signals survive costs, fast ones do not) can still be tested. Each characteristic is converted to its cross-sectional percentile rank each month, in [0, 1], with missing values set to 0.5, exactly as in the paper.

4. Methodology I: model inputs

Every portfolio method shares four inputs: trading costs Λ, investor wealth w, the covariance matrix Σ and, for the two-stage methods, ML return forecasts μ. We build each exactly as the paper and R code do, at smaller scale.

4.1 Timing and returns

A portfolio is formed at the end of month t using information dated t or earlier, and earns the excess return of month t+1:

$$r^{\pi}_{t+1} = \pi_t^{\top} r_{t+1}$$

Between rebalances, weights drift with returns. The portfolio inherited at t is last month's weights grown by each stock's total return relative to the market's, because the fund is assumed to grow with the market (Section 4.2):

$$\pi_{i,t}^{\text{start}} = \pi_{i,t-1}\,\frac{1+R_{i,t}}{1+R_{m,t}} \equiv g_{i,t}\,\pi_{i,t-1}$$

This bookkeeping (R `w_fun`; our `portfolio/accounting.py`) is shared by every method, so a timing error would affect all of them equally. It is tested against CRSP daily returns.

4.2 Trading costs and wealth

Price impact (paper Eq. 35). Calibrated to Frazzini, Israel and Moskowitz (2018): trading 1% of a stock's daily dollar volume V moves its price by 0.1%. With linear impact ½Λτ on a dollar trade τ, 0.001 = ½Λ·0.01V, so:

$$\Lambda_{i,t} = \frac{0.2}{V_{i,t}}, \qquad V_{i,t} = \text{average daily dollar volume over 126 days}$$

The dollar cost of trades τ is ½τ$^{\top}$Λτ. As a fraction of wealth, with τ = wΔπ:

$$\text{TC}_t = \frac{w_t}{2}\,(\pi_t - g_t\pi_{t-1})^{\top}\Lambda_t\,(\pi_t - g_t\pi_{t-1})$$

Example from the paper: trading $5M in a stock with $500M daily volume moves the price 0.1% and costs $5,000. Doubling the trade doubles the impact and quadruples the cost.

Wealth path. The fund grows with the market, $w_t = w_{t-1}(1 + R_{m,t})$, so it stays a constant share of market cap. We set w = $10B at the end of the sample and roll it backwards. The paper anchors at end-2020; we anchor at end-2025. We use the exact backward recursion $w_{t-1} = w_t/(1 + R_{m,t})$; the R code uses a first-order approximation that, over our 24 years, would understate early wealth (and so early costs) by 44%.

4.3 Covariance matrix (paper Eqs. 36–37)

Concept: a fundamental factor model. Estimating a full N × N covariance matrix from returns alone is noisy and need not be invertible. A Barra-style model instead fixes each stock's factor *exposures* from observable characteristics and estimates factor *returns* by regression. Stocks then co-move only through a small number of factors, plus independent idiosyncratic noise.

Each trading day d, a cross-sectional regression of stock returns on exposures X (industry dummies and standardised theme scores) gives that day's factor returns f:

$$r_{i,d} = X_{i,t}^{\top} f_d + \varepsilon_{i,d}$$

The covariance matrix is then factor risk plus idiosyncratic risk:

$$\Sigma_t = X_t\,F_t\,X_t^{\top} + \text{diag}\left(\sigma^2_{\varepsilon,i,t}\right)$$

F is an exponentially weighted covariance of daily factor returns over 10 years. An observation j days old gets weight proportional to 0.5^(j / half-life), with a half-life of 378 days for correlations and 126 days for variances (correlations move more slowly than volatilities). Idiosyncratic variance is an EWMA of squared residuals (half-life 126 days, at least 200 of the last 252 days). Daily values are multiplied by 21, so Σ is **monthly**.

Table 4.1 – Covariance model specification.

	Paper	Replication
Industry factors	12 Fama-French industries	6 sectors merged from FF12

	Paper	Replication
Theme factors	13 themes	6 themes
Regression universe	Portfolio universe (~650 stocks)	Top 500 by market cap (median 497 a day)
Window, half-lives, idiosyncratic rules	10 years; 378 / 126 / 126 days; 200 of 252	Same
Fit	Not reported	Mean daily R^2 0.17; 8,287 days

We regress on the top 500 rather than our 80 stocks because 12 regressors on 80 stocks a day would give unstable factor returns. **Calibration:** realised volatility is about 0.85 of predicted for universe portfolios (0.74 for single stocks), so the model over-predicts risk. Part of this is the × 21 scaling, which ignores the negative autocorrelation of daily returns; the R code does the same.

4.4 Return model for the two-stage methods

Markowitz-ML, Factor-ML and Static-ML need forecasts of next-month returns. As in the paper, these come from **random-feature ridge regression**.

Random Fourier features (paper Eqs. 38–39). Draw p/2 random vectors $w_j \sim N(0, g \cdot I)$ of the same length as the characteristic vector s (18 for us, 115 in the paper), and form:

$$RF(s_{i,t}) = \left[\sin(s_{i,t}^\top w_1),\ \cos(s_{i,t}^\top w_1),\ \ldots,\ \sin(s_{i,t}^\top w_{p/2}),\ \cos(s_{i,t}^\top w_{p/2})\right]$$

By Bochner's theorem (Rahimi and Recht, 2007), a linear model in these features approximates regression with a Gaussian kernel; a larger g means wigglier functions. Then fit by ridge:

$$\hat{\beta} = \arg\min_{\beta} \frac{1}{2n} \sum_{i,t} \left(r_{i,t+1} - \alpha - RF(s_{i,t})^\top \beta\right)^2 + \frac{\lambda}{2}\|\beta\|^2$$

Table 4.2 – Return-model specification.

	Paper	Replication
p (number of features)	2, 4, …, 512	Same
g (scale)	e^{-3}, e^{-2} (stated as a standard deviation; a variance in the R code)	Same values, as a variance (R)
λ (penalty)	0 and 100 values from e^{-10} to e^{10}	Same
Horizons	1 to 12 months (for Multiperiod-ML)	1 month only
Training sample	All US stocks, expanding window	All screened stocks (~3,700 a month), expanding window
Validation	Last 10 years, refit yearly	Last 5 years, refit yearly
Solver	`glmnet` (iterative)	Exact closed form: one eigendecomposition per (g, p, year) solves every λ at once

The closed form uses the eigendecomposition $X^TX = Q\,D\,Q^T$, so $\beta(\lambda) = Q\,(D + n\lambda I)^{-1}\,Q^T\,X^Ty$ for every λ with no new factorisation: 11 refits on 1.07 million stock-months take about 7 seconds.

Forecast quality. Across all screened stocks, the monthly rank correlation between forecasts and realised returns averages 0.07 in 2020–2025. Inside the 80-stock universe it averages 0.002. The paper does not report the equivalent statistic for its universe.

5. Methodology II: portfolio methods

All methods target the same objective, mean–variance utility net of trading costs, but differ in how much of it they see: the simple benchmarks see none of it, the two-stage methods see it after forecasting, and Portfolio-ML learns against it directly.

$$U = E\left[\pi_t^\top r_{t+1} - \tfrac{w_t}{2}\,\Delta\pi_t^\top \Lambda_t\,\Delta\pi_t - \tfrac{\gamma}{2}\,\pi_t^\top \Sigma_t\,\pi_t\right], \qquad \Delta\pi_t = \pi_t - g_t\pi_{t-1}$$

Base case for every method: γ = 10, w = $10B, monthly rebalancing, first month starting from value weights (as in R).

5.1 Benchmarks that ignore trading costs

Table 5.1 – Benchmark portfolios that ignore trading costs.

Method	Weights	Uses μ?	Uses Σ?	Paper vs ours
Market	Proportional to market cap	No	No	Same
1/N	1/N in each stock (DeMiguel et al., 2009)	No	No	Same
Minimum variance	$\Sigma^{-1}\mathbb{1} / (\mathbb{1}^\top\Sigma^{-1}\mathbb{1})$	No	Yes	Same
Markowitz-ML	$(1/\gamma)\,\Sigma^{-1}\mu$: the frictionless mean–variance optimum	Yes	Yes	Same
Factor-ML	Long the top group by μ, short the bottom group, value-weighted	Yes	No	Paper deciles; ours quintiles (~16 stocks a leg)
Rank-ML	Weights proportional to the rank of μ	Yes	No	Not replicated

Markowitz-ML is the textbook solution: it is the tangency portfolio scaled by risk aversion. In a frictionless world it maximises utility; with costs it trades far too much, which is the paper's point.

5.2 Static-ML and Static-ML*: costs added after forecasting

Static-ML (paper Eq. 34) solves the one-month problem with costs, taking the forecast μ as given. Setting the derivative of the one-period utility to zero gives:

$$\pi_t = \left(\gamma\Sigma_t + w_t\Lambda_t\right)^{-1}\left(\mu_t + w_t\Lambda_t\, g_t\, \pi_{t-1}\right)$$

The solution is a matrix-weighted average of the Markowitz portfolio and the inherited portfolio: expensive-to-trade stocks stay closer to where they were. It is "static" because it ignores the fact that today's position will still be held, and still need trading, next month.

Static-ML* (paper Eq. 41) adds a second tuning layer that rescales the inputs, chosen each December by cumulative realised utility:

Table 5.2 – Static-ML second-layer adjustments.*

Adjustment	Paper (text)	R code and ours
Shrink forecasts	$\mu^* = u\cdot\mu$, $u \in \{0.25, 0.5, 1\}$	Same
Inflate variances	$\Sigma^* = \Sigma + v\cdot\mathrm{diag}(\sigma)$, $v \in \{1, 2, 3\}$	$\gamma\Sigma + g\cdot\mathrm{diag}(\gamma\Sigma)$, $g \in \{0, 1, 2\}$ ($v = 1 + g$)
Scale costs	$\Lambda^* = k\cdot\Lambda$, $k \in \{1, ⅓, ⅕\}$	Same values

The second layer exists because two-stage portfolios are badly scaled out of sample: forecasts are noisier than the optimiser assumes, so it takes positions that are too large. Shrinking μ and inflating Σ corrects the scale. The paper's text says k "increases" costs, but its own grid {1, ⅓, ⅕} can only reduce them; we follow the grid.

5.3 Portfolio-ML: learning the weights directly

Step 1: the optimal policy has a known form. With quadratic costs, the paper shows (Proposition 3, building on Gârleanu and Pedersen, 2013) that the optimal portfolio trades part of the way towards an aim portfolio A each month:

$$\pi_t = m\, g_t\, \pi_{t-1} + (I - m)\, A_t$$

The trading-speed matrix m (Lemma 1) depends only on γΣ, wΛ and expected wealth growth, and is found by fixed-point iteration (10 iterations, as in R). If costs are zero, m = 0 and the investor jumps straight to the aim. As costs rise, m approaches I and the investor barely moves. Unrolling the recursion shows today's portfolio is a weighted sum of current and past aims:

$$\pi_t = \sum_{k=0}^{\infty} (m\, g)^k\, (I - m)\, A_{t-k}$$

Step 2: parameterise the aim, not the return. The aim is a discounted sum of expected returns at every future horizon, too hard to forecast directly. Portfolio-ML writes it as a linear function of transformed characteristics:

$$A_t = \mathrm{diag}(1/\sigma_t)\; S_t\, \beta, \qquad S_t = [\, \mathbf{c},\; RF(s_t)\,]$$

Each random-feature column is demeaned across stocks and scaled to unit sum of squares, and a scaled constant column c lets the model choose overall exposure. Dividing by each stock's volatility σ makes risky stocks get smaller positions. Without the scaling, positions would grow mechanically with the number of stocks.

Step 3: utility becomes quadratic in β. Substituting the aim into the unrolled policy, truncated at 12 months, makes the portfolio linear in β: $\pi_t = \Omega_t \beta$, where Ω_t combines the current and 11 lagged signal matrices weighted by powers of m·g. Realised utility in month t+1 is then:

$$u_{t+1}(\beta) = \beta^\top \tilde{r}_{t+1} - \tfrac{1}{2}\, \beta^\top \tilde{\Sigma}_t\, \beta$$

$$\tilde{r}_{t+1} = \Omega_t^\top r_{t+1}, \qquad \tilde{\Sigma}_t = \gamma\, \Omega_t^\top \Sigma_t \Omega_t + w_t\, (\Omega_t - g_t \Omega_{t-1})^\top \Lambda_t\, (\Omega_t - g_t \Omega_{t-1})$$

r̃ is the return of each signal portfolio. Σ̃is each signal's risk *plus its trading cost*: a signal that churns has a large cost term and is penalised. Maximising average realised utility with a ridge penalty has a closed form (paper Eq. 26):

$$\hat{\beta} = \left(\bar{\tilde{\Sigma}} + \lambda I\right)^{-1} \bar{\tilde{r}}$$

This is the paper's key trick: a dynamic portfolio problem with costs, solved by what is in effect a ridge regression on monthly sufficient statistics. Nothing after the statistics are built touches individual stocks, which is what makes 25 frontier re-runs affordable.

Step 4: tuning. Hyperparameters are p ∈ {64, 128, 256, 512}, g ∈ {e^{-3}, e^{-2}} and the 101-value λ grid. Each December, β is re-estimated for every combination from all months to date, and the combination with the highest cumulative realised utility since the validation start is used for the next year. Only one tuning layer is needed because the objective itself sets the portfolio's scale.

Where the R code differs from the paper and we follow the code: the lag weights are normalised by the inverse of their own truncated sum rather than multiplied by (I − m); the constant column is penalised by λ along

with the rest; Σ̃and r̃ are averaged rather than summed; and the random features carry no 1/√p factor, which the column scaling would undo anyway.

5.4 Walk-forward schedule

Table 5.3 – Walk-forward estimation schedule.

Item	Paper	Replication
Return model refit	Yearly from 1971, validation on last 10 years	Yearly for 2015–2025, validation on last 5 years
Static-ML* (u, v, k) picks	Each December from 1981, using utility since 1971	Each December from 2015, using utility since 2015
Portfolio-ML β	Yearly, from all months since 1952	Yearly, from all months since 2004
Portfolio-ML (p, g, λ) picks	Each December from 1981, using utility since 1971	Each December from 2015, using utility since 2015
Validation history behind the first test-year pick	10 years	4 years (2016–2019)
Look-ahead check	—	Unit test scrambles future returns; past picks unchanged

The paper's tuning had 10 years of validation results before its first out-of-sample year; ours had about 4, so our early test-window picks rest on less evidence.

6. Implementation and project organisation

The project separates logic, parameters and execution: every formula lives in one importable Python package (`src/jkmp/`), every number in one YAML file, and each pipeline stage is a thin script. The authors' R code sits read-only beside it as the answer key.

6.1 File tree

Figure 6.1 – Repository structure.

```
QF600_Replication/
├── CLAUDE.md                     project guide: status, conventions, commands
├── pyproject.toml                package + pinned dependencies (Python 3.11–3.13)
├── configs/
│   ├── base.yaml                 every design parameter (194 lines)
│   └── experiments/linear_pfml.yaml   overrides only the keys it changes
├── src/jkmp/                     ALL logic (~3,900 lines)
│   ├── config.py                 YAML → frozen, typed Config dataclasses
│   ├── data/
│   │   ├── wrds_pull.py          Stage 0: WRDS, JKP reference files, Ken French
│   │   ├── prepare.py            Stage 1: screens, timing columns, industries
│   │   └── universe.py           top-80 universe with rank buffer
│   ├── features.py               percentile ranks, theme scores
│   ├── costs.py                  Λ = 0.2 / dolvol, drift factor, wealth path
│   ├── covariance.py             Barra-style Σ (Eqs. 36–37)
│   ├── models/
│   │   ├── random_features.py    random Fourier features (Eqs. 38–39)
│   │   ├── ridge.py              closed-form ridge for a whole λ grid
│   │   └── return_model.py       yearly refit, next-month predictions
│   ├── portfolio/
│   │   ├── accounting.py         weight drift, trading cost, portfolio returns
│   │   ├── trading_speed.py      trading-speed matrix m (Lemma 1)
│   │   ├── recursion.py          month-by-month loop for path-dependent methods
│   │   ├── benchmarks.py         Market, 1/N, minimum variance
│   │   ├── ml_benchmarks.py      Markowitz-ML, Factor-ML
│   │   ├── static_ml.py          Static-ML, Static-ML* (Eqs. 34, 41)
│   │   ├── portfolio_ml.py       Portfolio-ML (Eqs. 35–37, Prop. 3)
│   │   └── grids.py              frontier grid over γ × wealth
│   └── evaluation/
│       ├── metrics.py            Table 2 statistics, utility flows
│       ├── frontier.py           Figures 1A, 1B
│       └── feature_importance.py Figure 5
├── scripts/                      thin runners: config → library → files
│   └── 00_download.py … 09_feature_importance.py
├── tests/                        18 pytest modules
├── report/make_report_extras.py  Table 3, Table 4, Figure 2 from saved outputs
├── docs/
│   ├── QF600 Replication Plan.md stage-by-stage plan and formulas
│   ├── JKMP R Code Map.md        where each paper equation lives in the R code
│   └── deviations.md             every difference from paper or R code
├── reference/jkmp_r/             authors' R code, read-only
├── data/raw/, data/processed/    git-ignored data (~1.4 GB)
└── outputs/                      tables, figures, logs
```

6.2 Pipeline stages

Each script reads the previous stage's files and writes its own, so a stage can be checked or rerun without redoing earlier ones.

Table 6.1 – Pipeline stages.

Script	Does	Reads	Writes
`00_download.py`	Pulls raw data	WRDS, French, JKP GitHub	`data/raw/*.parquet`, `jkp_usa_top500.csv`
`01_prepare.py`	Screens, ranks features, costs, wealth, universe	raw	`panel.parquet`, `wealth.parquet`, `universe_membership.parquet`
`02_covariance.py`	Daily factor regressions, EWMA Σ	panel, CRSP daily	`covariance/` (loadings, factor covariance, idiosyncratic variance)

Script	Does	Reads	Writes
`03_return_model.py`	Random-feature ridge, refit yearly	panel	`predictions.parquet`, `return_model/`
`04_benchmarks.py`	Market, 1/N, minimum variance, Markowitz-ML, Factor-ML	panel, Σ, predictions	`outputs/benchmarks/`
`05_static_ml.py`	Static-ML, Static-ML* with tuning	same	`outputs/static_ml/`, `table2_test.csv`
`06_pfml_inputs.py`	Portfolio-ML signal moments r̃, Σ̃ per month	panel, Σ	`data/processed/pfml/`
`07_portfolio_ml.py`	β, hyperparameter picks, weights	pfml moments	`outputs/portfolio_ml/`
`08_grids.py`	Re-tunes Static-ML* and Portfolio-ML at 25 (γ, wealth) points	all above	`outputs/grids/`, Figures 1A, 1B
`09_feature_importance.py`	Theme permutations	return models, Portfolio-ML fit	`outputs/feature_importance/`, Figure 5

6.3 How the Python maps to the authors' R code

Table 6.2 – Paper components, authors' R source and Python modules.

Paper component	R source (`reference/jkmp_r/`)	Python port (`src/jkmp/`)
Settings	`Main.R`	`configs/base.yaml`, `config.py`
Data preparation, screens	`1 - Prepare Data.R`	`data/prepare.py`, `data/universe.py`, `features.py`, `costs.py`
Covariance (Eqs. 36–37)	`3 - Estimate Covariance Matrix.R`	`covariance.py`
Random features, ridge	`0 - Return prediction functions.R` (`rff`), `2 - Fit Models.R`	`models/`
Weight drift, costs	`w_fun`, `pf_ts_fun`	`portfolio/accounting.py`
Trading speed m	`m_func`	`portfolio/trading_speed.py`
Static-ML(*)	`static_implement`, `sigma_gam_adj`	`portfolio/static_ml.py`
Portfolio-ML	`pfml_input_fun`, `pfml_search_coef`, `pfml_hp_reals_fun`, `pfml_aims_fun`, `pfml_w`	`portfolio/portfolio_ml.py`
Markowitz-ML, Factor-ML	`tpf_implement`, `factor_ml_implement`	`portfolio/ml_benchmarks.py`
Frontier	`6 - Implementable efficient frontier.R`	`portfolio/grids.py`, `evaluation/frontier.py`
Feature importance	`pfml_cf_fun`, `tpf_cf_fun`	`evaluation/feature_importance.py`

Paper component	R source (`reference/jkmp_r/`)	Python port (`src/jkmp/`)
Table 2 statistics	`pf_summary` in `6 - Base analysis.R`	`evaluation/metrics.py`

6.4 Engineering rules that protect the replication

- **Config, not code, for every parameter.** Runs are varied by overrides such as `investor.gamma=5`, so the frontier grid reuses the same code paths as the base case.
- **Follow the R code where the paper is ambiguous**, cite the R function in a comment, and log the choice in `docs/deviations.md`.
- **No explicit matrix inversion.** Linear systems use `solve` or Cholesky factorisation for numerical stability.
- **Timing conventions in one place.** `eom` is the formation month; `ret_ld1` is the next month's excess return; nothing dated after `eom` is used to form a portfolio at `eom`.
- **Tests guard the silent bugs.** `test_timing.py` checks return timing against CRSP daily data; `test_models.py` scrambles future returns and confirms past choices do not change (no look-ahead); `test_trading_speed.py` checks Lemma 1; other modules cross-check ported functions against R on small cases.

6.5 Pipeline at a glance

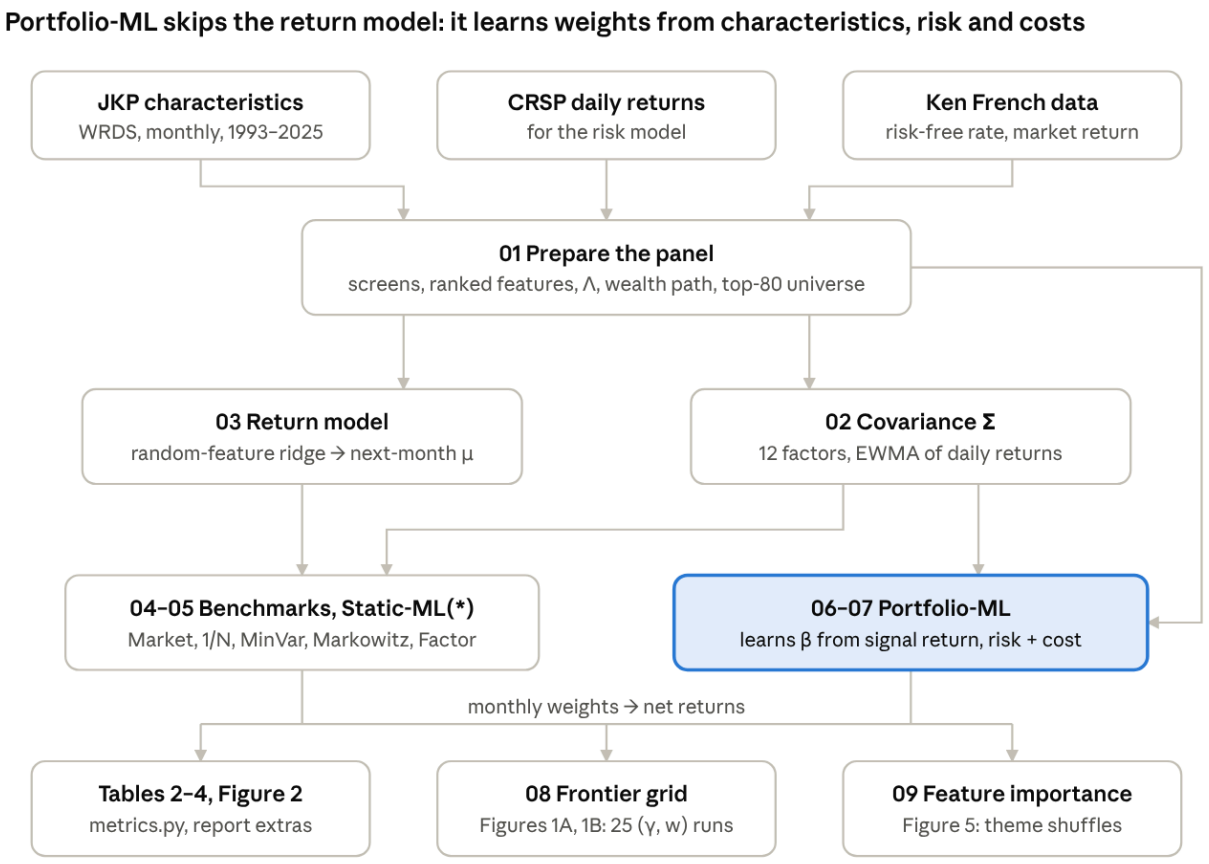


Figure 6.2 – Pipeline dependencies.

The two-stage benchmarks (left) need the return model's forecasts. Portfolio-ML (right) never uses them: it reads characteristics and Λ straight from the panel and Σ from the risk model, and learns its weights in one step.

7. Evaluation framework

We compute every statistic the paper reports, with the paper's definitions, on monthly out-of-sample returns, then annualise (means × 12, volatilities × √12). Realised utility is the headline metric because it is the objective every method is trying to maximise.

Table 7.1 – Performance metrics.

Metric	Definition	Reading it
R	Mean of $\pi_t^\intercal r_{t+1}$, the gross excess return	Return before costs
Vol	Standard deviation of gross returns	Risk taken
SR gross	R / Vol	Signal quality before costs
TC	Mean of $(w_t/2)\Delta\pi_t^\intercal \Lambda_t \Delta\pi_t$	Price impact paid, as a share of wealth
R − TC	Mean net return	What the investor keeps
SR net	(R − TC) / Vol	Implementable risk-adjusted return
Utility	$(R - TC) - (\gamma/2)\cdot Vol^2$, $\gamma = 10$	Certainty-equivalent net return; cash = 0
Turnover	Mean of $\Sigma_i \lvert \pi_{i,t} - g_{i,t}\pi_{i,t-1} \rvert$	Share of capital traded per month (not annualised)
Leverage	Mean of $\Sigma_i \lvert \pi_{i,t} \rvert$	Gross exposure (not annualised)

A code-versus-paper detail that affects every row. The paper's Eq. 11 defines realised utility with the variance of *net* returns. The R code (`pf_summary`) uses the variance of *gross* returns for Vol, SR net and Utility. We follow the code, so our figures are comparable with the published table, which the code produced. The difference is negligible for cost-aware methods, whose costs barely vary month to month.

Paper Table 3, probability of outperformance. For methods a and b, take monthly utility differences $d_t = u_{a,t} - u_b_{,t}$, where $u_t = r_t - TC_t - (\gamma/2)(r_t - \bar{r})^2$. Under a flat prior and normal errors, the posterior probability that a has higher mean utility is:

$$P(\bar{d} > 0) = \Phi\left(\frac{\bar{d}}{s_d/\sqrt{T}}\right)$$

It is approximately one minus the p-value of a one-sided t-test, so it depends heavily on T: 480 months in the paper, 72 for us.

Paper Table 4, correlations of monthly gross returns between methods.

Paper Figure 1, implementable efficient frontier. Re-run Static-ML* and Portfolio-ML at $\gamma \in \{1, 5, 10, 20, 100\}$, re-tuning every hyperparameter at each γ; plot each run's (Vol, R − TC). Markowitz-ML and Factor-ML are scaled to a range of target volatilities instead. Figure 1B repeats this across investor wealth.

Paper Figure 5, economic feature importance (Eq. 43). For each theme, randomly permute its characteristics across stocks within each month, re-run the method with all fitted parameters unchanged (β, hyperparameters, random draws), and measure the change in utility. Permuting a whole theme, not one feature, avoids substitution: if one value signal is shuffled, the others would still carry the information.

8. Results compared with the original study

Every replicated output reproduces the paper's ordering of methods and the collapse of cost-blind portfolios; none reproduces its profit levels. Paper figures are for 1981–2020; ours for January 2020 – December 2025. Both use a $10B investor with γ = 10.

8.1 Out-of-sample performance (paper Table 2)

Table 8.1 – Out-of-sample performance, replication (2020–2025) / original study (1981–2020). Annualised except turnover.

Method	R (ours / paper)	Vol	SR gross	TC	SR net	Utility	Turnover
Portfolio-ML	0.003 / 0.15	0.064 / 0.11	0.05 / 1.38	0.0017 / 0.006	0.03 / 1.33	**−0.018 / 0.086**	0.10 / 0.25
Static-ML*	−0.013 / 0.12	0.052 / 0.10	−0.26 / 1.15	0.0017 / 0.035	−0.29 / 0.81	−0.029 / 0.030	0.10 / 0.68
Static-ML	−0.022 / 0.26	0.109 / 0.28	−0.20 / 0.95	0.0042 / 0.044	−0.24 / 0.79	−0.085 / −0.165	0.18 / 0.80
Markowitz-ML	0.004 / 2.78	0.201 / 1.35	0.02 / 2.06	5.85 / +	−29.1 / –	−6.05 / –	3.95 / 79.36
Factor-ML	−0.040 / 0.11	0.186 / 0.13	−0.21 / 0.84	0.92 / 2.51	−5.16 / −18.09	−1.13 / −2.49	2.34 / 2.79
Minimum variance	0.052 / 0.11	0.146 / 0.11	0.36 / 1.03	0.17 / 0.82	−0.79 / −6.41	−0.22 / −0.77	0.76 / 1.35
1/N	0.124 / 0.10	0.169 / 0.17	0.73 / 0.57	0.0025 / 0.004	0.72 / 0.54	−0.022 / −0.051	0.07 / 0.07
Market	0.152 / 0.08	0.175 / 0.15	0.87 / 0.52	0.0002 / 0.000	0.86 / 0.51	−0.002 / −0.033	0.01 / 0.01

Source: `outputs/table2_test.csv`; paper Table 2. "+" and "–" are the paper's marks for extreme values.

- **Passive benchmarks match closely.** Market and 1/N have the same turnover as in the paper and similar risk; their higher returns reflect the 2020–2025 bull market.
- **Cost-blind methods collapse, as in the paper.** Markowitz-ML pays 585% a year in costs, Factor-ML 92%, minimum variance 17%. Costs are lower than the paper's because our universe is more liquid and smaller.
- **Cost-aware methods keep the paper's order.** Portfolio-ML has the highest utility, then Static-ML*, then Static-ML. Their costs are 3–20 times lower than in the paper.
- **Gross performance does not replicate.** Every ML method's gross Sharpe ratio is near zero or negative (Section 9).

8.2 Performance over time (paper Figure 2)

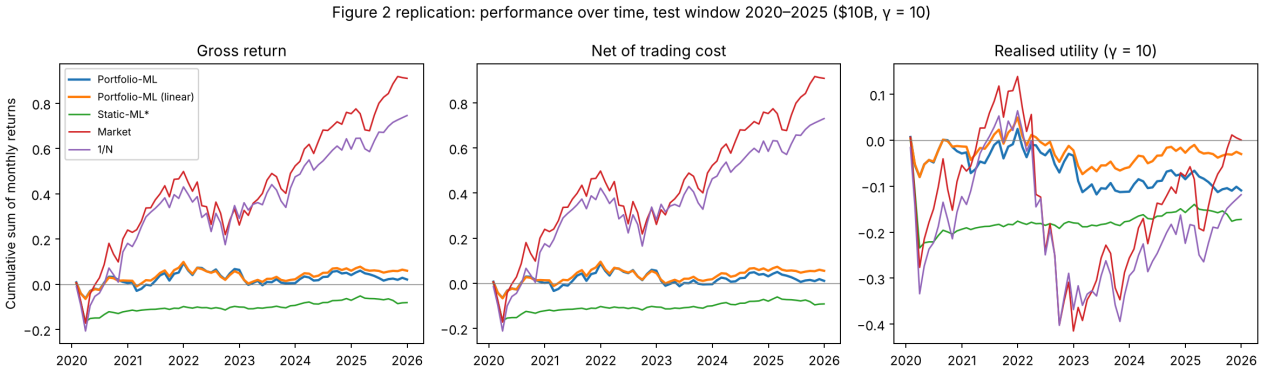


Figure 8.1 – Cumulative gross return, net return and realised utility, 2020–2025, $10B investor, γ = 10.

As in the paper, gross and net lines for the cost-aware methods are almost identical: costs are not what separates them. Unlike the paper, where Portfolio-ML rises steadily, our Portfolio-ML gains about 9% to early 2022, gives it back in the 2022 bear market, and drifts sideways. (The chart also shows linear Portfolio-ML, our extension.)

8.3 Statistical significance (paper Table 3)

Table 8.2 – Probability that the row method has higher mean utility than the column method, replication / original study.

Row beats column	Portfolio-ML (ours / paper)	Static-ML*	Factor-ML	Markowitz-ML
Portfolio-ML	—	64.7% / 99.9%	100.0% / 100.0%	100.0% / 100.0%
Static-ML*	35.3% / 0.1%	—	100.0% / 100.0%	100.0% / 100.0%
Factor-ML	0.0% / 0.0%	0.0% / 0.0%	—	100.0% / 100.0%

Source: `report/figures/table3_prob_outperform.csv`. The large gaps (cost-aware versus cost-blind) are as certain as in the paper. The small gap between Portfolio-ML and Static-ML* is not significant over 72 months.

8.4 Return correlations (paper Table 4)

Table 8.3 – Correlation of monthly gross returns between methods.

Pair	Ours	Paper
Portfolio-ML, Static-ML*	0.54	0.52
Static-ML*, Markowitz-ML	0.61	0.58
Factor-ML, Markowitz-ML	0.37	0.40
Portfolio-ML, Markowitz-ML	0.44	0.26
Portfolio-ML, Factor-ML	0.12	0.26
Static-ML*, Factor-ML	0.13	0.40

Source: `report/figures/table4_corr_gross.csv`. Correlations between the optimised methods are close to the paper's, which suggests the methods are built the same way. Factor-ML, which uses no covariance matrix and holds 16-stock quintile legs, is the least comparable.

8.5 Implementable efficient frontier (paper Figure 1A)

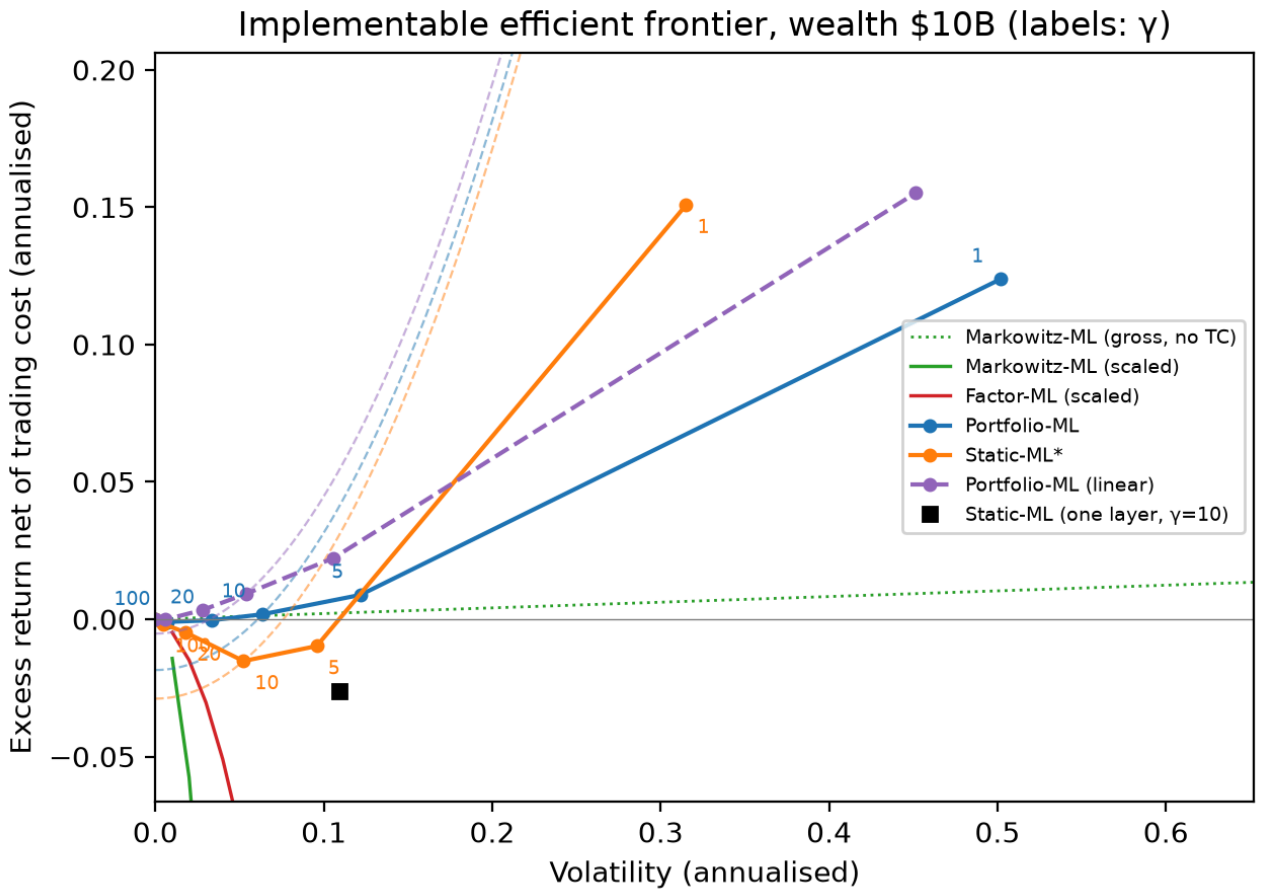


Figure 8.2 – Implementable efficient frontier at $10B wealth; one point per risk aversion γ ∈ {1, 5, 10, 20, 100}.

- **Matches:** Markowitz-ML and Factor-ML frontiers dive below zero as soon as they leave cash, and no method gets near the no-cost Markowitz line.
- **Matches:** Portfolio-ML lies above Static-ML* at γ = 5, 10 and 20.
- **Differs:** at γ = 1 Static-ML* earns more (15% net at 32% volatility vs 12% at 50%); the paper's Portfolio-ML dominates throughout.
- **Differs:** our frontiers are close to straight lines because costs in this liquid universe stay below 2% a year even at γ = 1; the paper's bend clearly.

The dashed purple line, linear Portfolio-ML, is our extension and not part of the paper.

8.6 Frontier by investor size (paper Figure 1B)

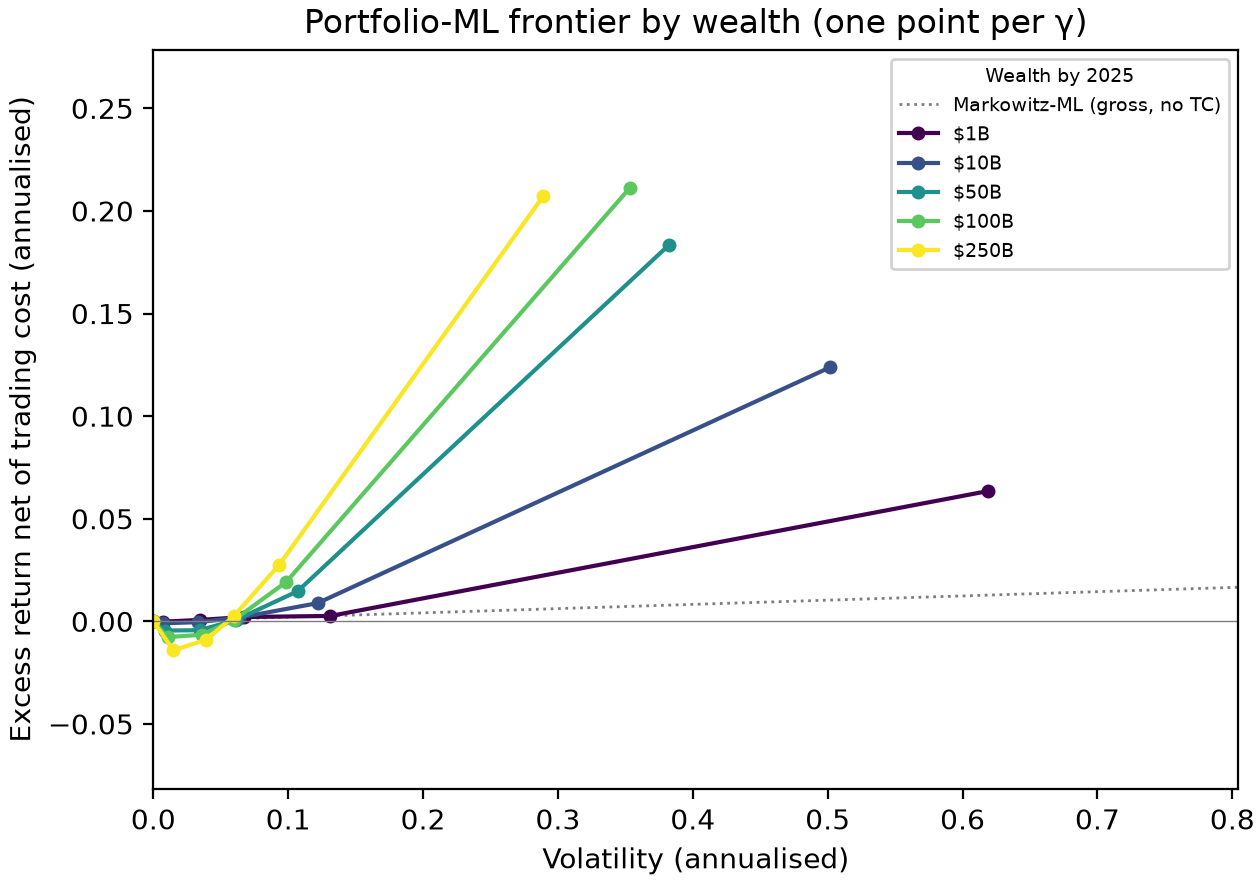


Figure 8.3 – Portfolio-ML implementable frontier by end-of-sample wealth, $1B to $250B.

Opposite to the paper. The paper's frontier contracts as wealth grows. Ours expands at low risk aversion (γ = 1 net return rises from 6% at $1B to 21% at $250B) and is almost flat at γ = 10 (utility −0.020 to −0.015). Section 9.3 gives a likely explanation.

8.7 Economic feature importance (paper Figure 5)

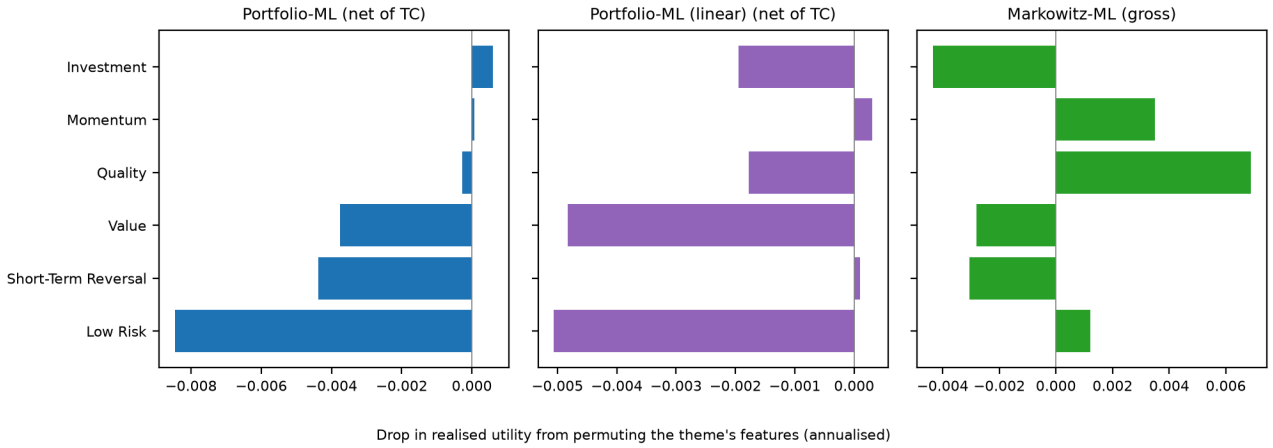


Figure 8.4 – Change in realised utility when each theme's characteristics are permuted (negative = the theme adds utility).

Table 8.4 – Most important themes, original study versus replication.

	Paper	Replication
Portfolio-ML, most important themes	Quality, momentum, investment	Low risk (−0.85% a year), short-term reversal (−0.44%), value (−0.38%)
Portfolio-ML, short-term reversal	Unimportant after costs	Second most important
Markowitz-ML (gross), most important	Quality, short-term reversal	Investment, reversal, value; shuffling quality *improves* utility

The paper's finding does not replicate. The effects are fractions of a percent, and with 72 months they depend on the random shuffle: across nine seeds only Portfolio-ML's low risk and reversal, and Markowitz-ML's quality, keep their sign. The middle panel (linear Portfolio-ML) is our extension.

9. Discussion: why the results differ from the original study

The most likely reason is that our setup gives the method almost no return signal to work with. The code and the method look sound, but there was very little for any strategy to profit from.

9.1 Evidence that the signal, not the code, is the problem

The clearest test is Markowitz-ML *before* trading costs. It ignores costs entirely, so if the forecasts were good it would look strong on paper even if it collapsed after costs. Its gross Sharpe ratio is 2.06 in the paper and 0.02 in our replication. The forecasts fail before costs enter, and every other ML method starts from the same forecasts or the same characteristics.

Meanwhile, the results that do not depend on signal strength replicate:

- The market portfolio's turnover (1.3% a month) and near-zero trading cost come out as they should, confirming the timing and cost bookkeeping.
- Cost-blind methods collapse after costs, as in the paper.
- The ranking Portfolio-ML > Static-ML* > Static-ML in utility matches the paper.
- Return correlations between methods are close to the paper's: Portfolio-ML with Static-ML* is 0.54 against 0.52, and Static-ML* with Markowitz-ML is 0.61 against 0.58.

That pattern points to weak inputs rather than a bug.

9.2 Likely causes, most important first

Table 9.1 – Likely causes of the divergence, most important first.

#	Cause	Paper	Replication	Evidence
1	Universe	~650 NYSE stocks above median size, reaching into mid-caps	80 largest US stocks	Forecast rank correlation 0.07 across all ~3,700 screened stocks, 0.002 inside our 80
2	Features	115 characteristics, 13 themes	18 characteristics, 6 themes	Paper's most valuable themes after costs (quality, momentum, investment) carried by only 3 features each
3	Test window	480 months, 1981–2020	72 months, 2020–2025, after publication	Table 3: 65% vs 99.9% for Portfolio-ML over Static-ML*

#	Cause	Paper	Replication	Evidence
4	Tuning history	10 years of validation before the first test year	About 4 years	Portfolio-ML picked the fewest features allowed from 2018 on; its ridge penalty rose fourteenfold
5	Risk model	Not reported	Realised volatility 0.85 of predicted	Portfolio-ML runs at 6.4% volatility vs 11% in the paper

1. The universe is only 80 mega-caps. Across all ~3,700 stocks that pass the screens, the forecasts have a monthly rank correlation with realised returns of 0.07, normal for this literature. Inside our 80 stocks it is 0.002, and in 2025 it is −0.11. This fits the paper's own argument that ML predictability lives mostly in smaller, less liquid stocks. The most heavily traded and analysed stocks in the world are where mispricing is least likely to persist.

2. We use 18 characteristics instead of 115. The paper finds quality, momentum and investment most valuable after costs. With only three characteristics per theme, much of that information is missing, and the random-feature model has fewer interactions to discover.

3. The test window is short and unusual. With 72 months, small utility gaps cannot be told apart from noise: the posterior probability that Portfolio-ML beats Static-ML* is 65%, against 99.9% over the paper's 480 months. The period also contains a pandemic crash, the fastest Fed hiking cycle in decades and a mega-cap-led bull market in which the market alone had a Sharpe ratio of 0.86. Long–short tilts within the largest stocks had little room to add value. And the window falls entirely after the paper's sample, when published anomalies tend to weaken (McLean and Pontiff, 2016).

4. Tuning had less history to learn from. Portfolio-ML's hyperparameters for the first test year rest on about 4 years of validation results, against 10 in the paper; the return model validates on 5 years rather than 10. The choices it made show the strain: from 2018 on, Portfolio-ML always selected the fewest random features allowed ($p = 64$) and a ridge penalty that rose from 15 to 211. In effect it kept shrinking its bets towards zero, which is what a well-behaved model should do when it finds no signal.

5. The risk model overstates risk. Realised volatility is about 0.85 of predicted for universe portfolios, partly because scaling daily variance by 21 ignores the negative autocorrelation of daily returns (the R code does the same). The optimisers therefore hold smaller positions than they should. This mainly affects the size of the bets, not whether they pay.

9.3 Why larger investors did better (Figure 1B)

In the paper, more wealth means higher costs and a worse frontier. In our data the signal is mostly noise, and higher costs act like regularisation: they push the trading-speed matrix m towards I, so each month's portfolio averages the aim over more past months, smoothing out noise in β and the signals. With a strong signal, costs only subtract value; with a weak one, the forced patience can add it. This explanation is consistent with the data but has not been tested directly.

9.4 How to tell which cause dominates

Each rerun changes one factor; the config-driven design makes them cheap to set up.

Table 9.2 – Diagnostic reruns to isolate the main cause.

Test	Change	If performance recovers
Universe	Rerun 2020–2025 on the top 500 stocks	The universe is the main cause

Test	Change	If performance recovers
Time period	Rerun on the current universe with a pre-2020 test window, e.g. 2010–2019	The post-2020 period is the main cause; if it still fails, the period is not
Features	Add more of the paper's 115 characteristics, especially quality, momentum and investment	The feature set is the main cause

10. Conclusion

This project replicated the main empirical analysis of Jensen et al. (2026) in Python, on a universe of 80 US mega-caps with 18 characteristics, over a test window (2020–2025) that begins where the paper's ends.

The replication supports the paper's methodological claims. Trading costs make cost-blind ML portfolios unimplementable for a $10B investor; building costs into the optimisation removes almost all of that cost; and among cost-aware methods, Portfolio-ML, with a single tuning layer, delivers the highest utility, ahead of the two-layer Static-ML*. The implementable efficient frontier has the shape the paper describes, and the correlations between methods are close to those reported.

The replication does not support the paper's quantitative claims in this setting. No ML method earns a meaningful net return, Portfolio-ML's advantage over Static-ML* is not statistically significant over 72 months, the frontier does not contract with investor size, and the most valuable characteristics differ. Section 9 attributes this mainly to the universe: return predictability, which the paper itself locates in smaller and less liquid stocks, is close to zero among the 80 largest. A shorter tuning history, a smaller feature set and an unusual post-publication test period contribute.

The central lesson of the paper therefore survives: portfolio methods should be judged net of trading costs, and learning with costs in the objective is superior to adding them afterwards. Whether that lesson translates into the paper's level of profit depends on the strength of the underlying signal, which the diagnostic reruns in Section 9.4 are designed to isolate.

References

- DeMiguel, V., Garlappi, L. and Uppal, R. (2009). Optimal Versus Naive Diversification: How Inefficient Is the 1/N Portfolio Strategy? *Review of Financial Studies* 22(5): 1915–1953.
- Frazzini, A., Israel, R. and Moskowitz, T. J. (2018). Trading Costs. Working paper.
- Gârleanu, N. and Pedersen, L. H. (2013). Dynamic Trading with Predictable Returns and Transaction Costs. *Journal of Finance* 68(6): 2309–2340.
- Jensen, T. I., Kelly, B. and Pedersen, L. H. (2023). Is There a Replication Crisis in Finance? *Journal of Finance* 78(5): 2465–2518.
- Jensen, T. I., Kelly, B., Malamud, S. and Pedersen, L. H. (2026). Machine Learning and the Implementable Efficient Frontier. *Review of Financial Studies* 39(10): 3035–3078. doi:10.1093/rfs/hhag022
- Jensen, T. I., Kelly, B., Malamud, S. and Pedersen, L. H. Replication code. github.com/theisij/ml-and-the-implementable-efficient-frontier
- McLean, R. D. and Pontiff, J. (2016). Does Academic Research Destroy Stock Return Predictability? *Journal of Finance* 71(1): 5–32.
- Rahimi, A. and Recht, B. (2007). Random Features for Large-Scale Kernel Machines. *Advances in Neural Information Processing Systems* 20.

- Data: Wharton Research Data Services (JKP Global Factor Data; CRSP), Kenneth R. French Data Library.

Appendices

A. Deviation log

Every difference from the paper or the R code, with its config key and code location, is in `docs/deviations.md`. The main ones:

Table A.1 – Main deviations from the paper and the authors' R code.

#	Area	Paper / R code	Replication	Reason
D1	Exchanges	NYSE only	NYSE + Nasdaq	Sample starts after Nasdaq's 1972 entry into CRSP
D2–D3	Universe	Above-median NYSE market cap; 12-month entry/exit rule	Top 80; 70/100 rank buffer, reset each December	Compute grows with N^2
D4	Sample	1952–2020; test 1981–2020	1993–2025; test 2020–2025	Compute; post-publication test
D5–D6	Features, risk factors	115 features, 13 themes, 12 industries	18 features, 6 themes, 6 sectors	Compute, interpretability
D7	Covariance regression	Portfolio universe	Top 500 stocks	80 stocks too few for a stable daily regression
D10	Return horizons	1–12 months	1 month	Multiperiod-ML out of scope
D11	Daily returns	`usa_dsf.csv` (not public)	CRSP v2 `dsf_v2`	Availability
D13	Risk-free timing in total return	Formation month (one month stale)	Return month	Accuracy; tested against CRSP
D14	Wealth roll-back	First-order approximation	Exact recursion	Approximation understates 2001 wealth by 44%
D19	Exits from universe	Free	Charged at that month's cost unless delisted	Realism; adds at most 0.35% a year
D21	Ridge solver	`glmnet`, iterative	Exact closed form	Same objective, exact and fast
D22	Random draws	R's generator	NumPy's generator	Draws cannot match R's
D23	Return-model validation	10 years	5 years	Shorter history
D25	Factor-ML legs	Deciles	Quintiles	~8 stocks a decile leg would be too few
D26–D29	Tuning schedule	Validation from 1971, picks from 1981	Validation and picks from 2015	Predictions start in 2015
D30	Frontier wealth grid	≈ 0, $1B, $10B, $100B	$1B, $10B, $50B, $100B, $250B	Brackets the base case where costs bind
D31	Feature importance window	1981–2020	2020–2025	Test window as for every result

B. Where we follow the R code rather than the paper's text

Table B.1 – Points where the replication follows the R code rather than the paper's text.

Topic	Paper text	R code (followed)
Random-feature scale	η is a standard deviation	g is a variance
Random-feature normalisation	1/√p factor	None (column scaling undoes it)
Portfolio-ML lag weights	Multiply by (I − m)	Normalise by the inverse of their truncated sum
Portfolio-ML ridge	Penalises signal coefficients	Also penalises the constant; uses means, not sums
Covariance units	—	Daily × 21 = monthly (comment says "annualise")
Table 2 Vol, SR net, Utility	Net-return variance (Eq. 11)	Gross-return variance
Static-ML* variance add-on	$\Sigma + v \cdot \mathrm{diag}(\sigma)$, $v \in \{1, 2, 3\}$	$\gamma\Sigma + g \cdot \mathrm{diag}(\gamma\Sigma)$, $g \in \{0, 1, 2\}$
Static-ML* cost scaling	Divide Λ by φ	Multiply Λ by $k = 1/\varphi$
Trading speed m	Exact fixed point	10 iterations (residual ~10^{-4})
First month	—	Start from value weights
Feature ranks	Rank mapped to [0, 1]	Ties take the maximum rank; exact zeros ranked 0; missing 0.5

C. Not replicated

Multiperiod-ML and Multiperiod-ML* (need 12 return horizons), Rank-ML, Figure 3 (positions by liquidity), Figure 4 (example stock weights), short-selling fees (paper Section 6.4), performance across the size distribution (paper Section 6.5), and the simulation study.